AI Chatbot on AWS

Project documentation: from research to deployment

Generative AI web application on an AWS EC2 web server (Level 3)

Author Md Humayun Kabir (K@BIR)

Batch Kombo OCT25, Jensen Education

Date 14 October 2026

Live site https://askforhelp.duckdns.org

Contents

1. Project overview 3

1.1 How the project meets the requirements 3

1.2 Technology stack 3

1.3 Architecture 4

2. Phase 1: Research (pre-study) 4

2.1 Information gathering 4

2.2 Choice of tools and techniques 4

2.3 Planning 5

2.4 Problems and solutions in the research phase 5

3. Phase 2: Starting the work (implementation) 6

3.1 How the work started: AWS setup 6

3.2 Structure of the project and build order 6

3.3 How the functions were developed 6

3.4 Deployment on the server 7

3.5 Problems and solutions in the implementation phase 7

4. Phase 3: Finishing the work (completion and improvement) 9

4.1 Completing the solution 9

4.2 Testing the whole solution 9

4.3 Improvements and optimisation 10

4.4 Publication: domain name and HTTPS 10

4.5 Problems and solutions in the finishing phase 11

5. Phase 4: Summary of problems and solutions 13

6. Publishing the project on GitHub 14

7. Conclusion 15

What I achieved 15

What I learned 15

What could be improved 15

1. Project overview

The goal of this assignment was to plan, build and deploy an application that uses Generative AI and runs in the AWS cloud, using **Level 3**: building a web server. I chose to build an **AI chatbot**. Every answer the user sees is written by a large language model running on **AWS Bedrock**, so the AI is the core of the solution and not an add-on. The chatbot runs on a Linux web server (EC2) and is reachable from any browser at **https://askforhelp.duckdns.org**, a free domain name that points to the server's public IP address, with a trusted HTTPS certificate.

1.1 How the project meets the requirements

Requirement	How it is met
Uses Generative AI	A Claude model on AWS Bedrock (Converse API) generates every reply.
AI is a central part	The whole application is a chat interface to the model; the model also decides when to call live tools (web search, weather, current time).
Hosted in AWS	EC2 instance in region eu-north-1 (Stockholm); the AI runs on Bedrock in the same region.
Level 3: web server	Ubuntu server with nginx (reverse proxy), gunicorn (application server) and a Flask application, started automatically by systemd.
Reachable online	Public (Elastic) IP address 16.170.76.172, and the domain name https://askforhelp.duckdns.org that points to it.
Secure connection (extra)	HTTPS with a free Let's Encrypt certificate; the browser shows the padlock.

1.2 Technology stack

Layer	Technology	Why
AI model	AWS Bedrock, Claude model through an EU inference profile	Stays inside AWS; no separate API key; one API (Converse) for chat and tools.
Backend	Python 3, Flask, boto3	Small, readable, easy to deploy; boto3 is the official AWS SDK.
Frontend	One HTML file with CSS and JavaScript	No build step; served directly by Flask.
Web server	nginx + gunicorn	Standard production setup instead of the Flask development server.
Process control	systemd	Starts the app on boot and restarts it if it crashes.
Security	IAM role on the EC2 instance, security group	No access keys stored on the server; only ports 22, 80 and 443 are open.
Domain + HTTPS	DuckDNS (askforhelp.duckdns.org), Let's Encrypt with certbot	Free domain name and a free, trusted certificate that renews automatically.
Live data	Open-Meteo API, DuckDuckGo search (ddgs)	Free and need no API key.

1.3 Architecture

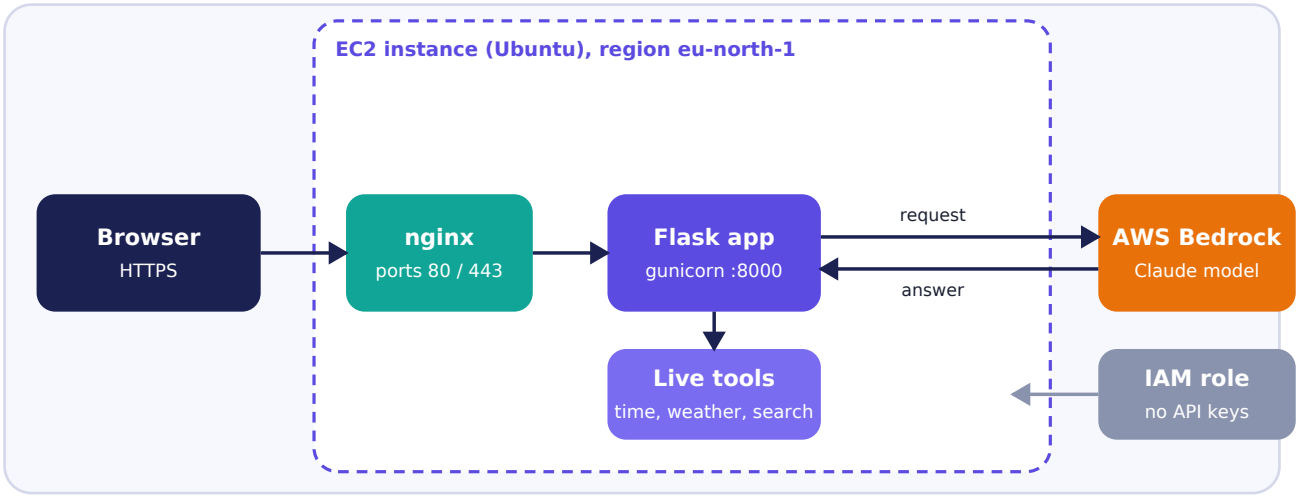


Figure 1. The browser reaches the server through the domain name askforhelp.duckdns.org over HTTPS. nginx ends the HTTPS connection and forwards to the Flask application, which calls the model on Bedrock using the instance's IAM role.

The browser keeps the conversation history and sends it with every message, so the server is stateless and needs no database. When the model decides it needs fresh information it asks the Flask app to run a tool; the result is sent back to the model, which then writes the final answer.

2. Phase 1: Research (pre-study)

2.1 Information gathering

- **The assignment instructions** defined the goal: a GenAI function, hosting in AWS, and Level 3 (web server).
- **Project ideas** from the instruction list (chatbot, document assistant, image generator, ...). I picked a chatbot because it is the clearest way to show GenAI as the main feature and is realistic to finish and deploy.
- **AI services** listed in the assignment (OpenAI, Anthropic Claude, Google Gemini, AWS Bedrock, Hugging Face, local models) and AWS documentation for Bedrock, IAM roles, EC2 and security groups.
- **Practical testing** in the AWS console: checking which models and inference profiles were visible in region eu-north-1 before writing code.

2.2 Choice of tools and techniques

Decision	Chosen option	Reason
Application type	AI chatbot	Clear use case; GenAI is the whole product.
AI provider	AWS Bedrock (Claude)	Everything stays in AWS; the EC2 role gives access without secret keys; no extra account or billing outside AWS.
Region	eu-north-1 (Stockholm)	Closest region; the same region for server and AI.
Hosting	EC2 web server (Level 3)	Required by the assignment; full control of the stack.
Backend	Python + Flask	Fast to build and easy to explain.
Serving	nginx + gunicorn + systemd	Production-style and automatically restarted.

2.3 Planning

I split the work into small steps so that each one could be tested on its own before the next one started:

- Prepare AWS: choose the region, check Bedrock model access, create an IAM role with Bedrock permission.
- Launch an EC2 instance (Ubuntu) with a security group for SSH and HTTP, and give it a public IP.
- Build the Flask backend with one chat endpoint that calls Bedrock.
- Build the chat page (HTML, CSS, JavaScript).
- Install the project on the server and run it with gunicorn and systemd.
- Put nginx in front so the site works on the normal web port 80.
- Add a free domain name and HTTPS so the site opens securely.
- Test everything from a browser through the public IP, then improve it.
- Write the documentation and publish the code on GitHub.

2.4 Problems and solutions in the research phase

Many possible solutions, hard to choose	
Problem	The assignment allowed many AI services, project types and hosting options, so it was unclear where to start.
Cause	Too many open choices and no comparison criteria.
Solution	I compared the options on three criteria: stays inside AWS, no secret keys to manage, and simple to deploy. Bedrock with an EC2 IAM role was the best match, and a chatbot was the simplest project type that still makes the AI the central feature.

Unclear requirement: what does "Level 3" mean?	
Problem	The task only said to "use Level 3 to build a web server".
Cause	The requirement was short and I did not know how much was expected.
Solution	I broke it into concrete parts: a Linux server in AWS, a web server software (nginx + gunicorn) and an application reachable on a public IP. Each part became a step in the plan.

Not sure which models were available in my region	
Problem	Bedrock model names differ per region and some models need an "inference profile" ID.
Cause	Model availability depends on region and account; documentation lists many IDs.
Solution	I decided to verify the model in the AWS console (Bedrock model catalog and cross-region inference) and test a real call early, before building the rest. This avoided building on a model that I could not use (see section 3.5, problem 5).

3. Phase 2: Starting the work (implementation)

3.1 How the work started: AWS setup

- **Region:** I set the console to Europe (Stockholm) eu-north-1 and kept it for everything.
- **IAM role:** I created the role `chatbot-ec2-role` for the EC2 service with a small inline policy that only allows `bedrock:InvokeModel` and `bedrock:InvokeModelWithResponseStream`. With a role the code needs no access keys, which is both simpler and safer.
- **EC2 instance:** Ubuntu server with the IAM role attached. The security group allows SSH (22) and HTTP (80); port 443 was added later for HTTPS.
- **Public IP:** an Elastic IP was attached so the address stays the same.
- **Connecting:** I used EC2 Instance Connect, a terminal in the browser, so no key file was needed.

3.2 Structure of the project and build order

```
chatbot/
  app.py            Flask backend: chat endpoint, Bedrock call, tools
  index.html        Chat page (HTML + CSS + JavaScript)
  requirements.txt  Python packages
  deploy/           systemd service and nginx configuration
  docs/             This documentation
```

The parts were created in this order, each one tested before moving on:

- **1. Backend skeleton** with a `/health` route to check that the server runs.
- **2. Chat endpoint** `POST /api/chat` that validates the conversation and calls the Bedrock Converse API.
- **3. Chat page** that sends the history and shows the reply.
- **4. Deployment** on EC2: virtual environment, gunicorn, systemd service, nginx.

3.3 How the functions were developed

Chat endpoint. The server checks that the history is a list of user/assistant messages, limits its length and the size of each message, and makes sure the conversation ends with a user message. Invalid input returns a clear error (HTTP 400). Errors from Bedrock return a friendly message (HTTP 502) and are written to the log instead of crashing the app.

```
response = bedrock.converse(
    modelId=MODEL_ID,                # set with the BEDROCK_MODEL_ID variable
    system=[{"text": system_prompt()}],
    messages=messages,
    toolConfig=TOOL_CONFIG,          # added later for live information
    inferenceConfig={"maxTokens": 1000, "temperature": 0.5},
)
```

Configuration through environment variables. Region, model ID, time zone and system prompt are environment variables with safe defaults. This made it possible to change the model without editing code, which turned out to be important when the first model was not available.

Frontend. The page keeps the history in JavaScript, shows a "..." placeholder while waiting, writes the answer with `textContent` (this prevents HTML injection) and removes the last message on an error so the user can retry.

3.4 Deployment on the server

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
sudo systemctl daemon-reload
sudo systemctl enable --now chatbot      # gunicorn on 127.0.0.1:8000
sudo nginx -t && sudo systemctl restart nginx   # public port 80 -> 8000
```

The systemd service (`deploy/chatbot.service`) runs gunicorn as the `ubuntu` user, sets the environment variables and restarts the app automatically. nginx (`deploy/nginx-chatbot.conf`) forwards port 80 to gunicorn.

3.5 Problems and solutions in the implementation phase

1. EC2 Instance Connect could not connect: no public IP	
Problem	The Connect page showed: "*No public IPv4 or IPv6 address assigned. With no public IPv4 or IPv6 address, you can't use EC2 Instance Connect.*"
Cause	The instance had been launched without "auto-assign public IP", so it was not reachable from the internet at all (neither by the terminal nor later by a browser).
Solution	I allocated an **Elastic IP** and associated it with the instance. After that the Connect button worked and the same address became the public address of the website. I also checked that the security group had inbound rules for SSH (22) and HTTP (80).

2. Files were corrupted when pasting commands into the terminal	
Problem	Installing packages failed with: `ERROR: Invalid requirement: "EOFcat > requirements.txt << 'EOF'"`, and the app crashed with `NameError: name 'EOFcat' is not defined`.
Cause	I pasted several multi-line blocks (`cat > file << 'EOF'`) one after another without waiting for the prompt. The closing `EOF` of one block was glued to the first command of the next, so junk text ended up inside the files.
Solution	I repaired the files with small commands: `printf` to rewrite requirements.txt, and `sed -i '89,$d' app.py` to delete everything after the last real line. I then added checks after each paste (`tail -n 3`, `python3 -m py_compile app.py`, `ls -l`) and changed my routine to paste one block at a time and wait for the prompt.

3. Python packages were missing (boto3)	
Problem	`ModuleNotFoundError: No module named 'boto3'` when starting the app.
Cause	`pip install` had failed because of the corrupted requirements.txt (problem 2), so nothing was installed.
Solution	After fixing requirements.txt I ran `.venv/bin/pip install -r requirements.txt` again. It finished with "Successfully installed ... boto3 ... flask ... gunicorn" and the app started.

4. Environment variable not set because of angle brackets	
Problem	`export BEDROCK_MODEL_ID=<eu.anthropic...>` gave `No such file or directory` and `syntax error near unexpected token`.
Cause	I typed the < > signs from the example. In the shell < means "read from a file". The app then started with an old value.
Solution	I removed the brackets (they only mark where to put a value) and checked the result with `echo $BEDROCK_MODEL_ID` before starting the app again.

5. Bedrock rejected the model: invalid identifier, then access denied	
Problem	First: `ValidationException: The provided model identifier is invalid.` Later: `AccessDeniedException: anthropic.claude-haiku-5-5 is not available for this account.`
Cause	The first ID did not exist in the account and region (older models are retired and newer ones need an inference-profile ID). The second model existed but was not enabled for my account.
Solution	I looked in the Bedrock console (Model catalog and Cross-region inference), used the exact **inference profile ID** `eu.anthropic.claude-haiku-4-5-20251001-v1:0`, and tested it directly with `curl`. It returned a normal reply, so I set it in the systemd service as `BEDROCK_MODEL_ID`. Because the model ID is an environment variable, no code change was needed.

```
curl -s -X POST localhost:8000/api/chat -H 'Content-Type: application/json' \
  -d '{"messages":[{"role":"user","content":"Say hello in one sentence"}]}'
-> {"reply":"Hello! I'm here to help you ..."}
```

4. Phase 3: Finishing the work (completion and improvement)

4.1 Completing the solution

After the model worked I put nginx in front of gunicorn, opened the public IP in a browser and sent the first real messages. The chatbot answered correctly, so the basic requirements (GenAI, AWS, web server, public IP) were met.

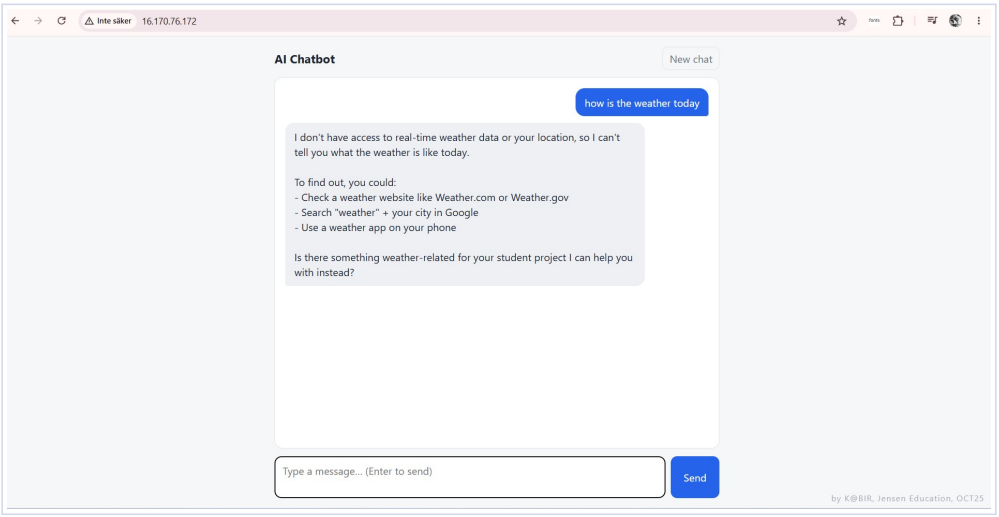


Figure 2. First working version through the public IP. The answer also revealed two things to improve: no real-time data and a wording from my system prompt ("student project").

4.2 Testing the whole solution

Test	How	Result
Server health	Open /health or curl localhost:8000/health	Returns status ok and the model ID.
Model call	curl POST /api/chat on the server	Returns a JSON reply from Bedrock.
Browser chat	Open https://askforhelp.duckdns.org and send messages	Answers appear; history is kept in the chat.
Input validation	Send an empty or invalid request	HTTP 400 with a clear error.
Tool loop (logic)	Test with a simulated model before deploying	Tools run, errors are reported, the loop stops after the maximum number of rounds.
Domain and HTTPS	Open https://askforhelp.duckdns.org	Opens without problems; the connection is secure and the chat works.
Real-time questions	Ask about weather, date and time, news	Answered using the live tools.
Always-on	Service managed by systemd	App is started by systemd and restarted automatically.

4.3 Improvements and optimisation

Live information with tool use. A language model only knows what it learned in training, so it cannot know today's weather or news. I added three tools that the model can call through the Bedrock Converse tool-use feature:

- `get_current_time`: the current date and time (Stockholm time by default).
- `get_weather`: current weather and a 3-day forecast from the free Open-Meteo service.
- `web_search`: web search through DuckDuckGo for news, prices and recent facts.

The application sends the tool definitions to the model. If the model answers with a tool request, the server runs the tool, returns the result to the model, and repeats this for at most four rounds before the final answer is produced. Tool errors are returned to the model as errors, so the chatbot can say honestly that something failed instead of crashing. The current date and time is also added to the system prompt on every request.

Better system prompt. I removed the "student project website" sentence, so the assistant behaves as a general helper, answers in the language of the user and mentions its sources when it searches the web.

Design and branding. The page now has a gradient background, a frosted-glass chat card, gradient message bubbles, a light and a dark theme, the tagline "askforhelp by K@BIR" and a large faded "askforhelp by K@BIR" text behind the conversation, plus the watermark "by K@BIR, Jensen Education, OCT25". The layout works on narrow screens too.



Figure 3. Final interface (light theme).

4.4 Publication: domain name and HTTPS

At first the site was only available over plain HTTP on the public IP address (the browser showed "Not secure"). To make it professional and safe to use, I added a free domain name and a trusted HTTPS certificate:

- **1. Domain name:** I created the free sub-domain `askforhelp` at DuckDNS and pointed it to the Elastic IP address of the server. I checked it on the server with `getent hosts askforhelp.duckdns.org` until it returned the IP.
- **2. Port 443:** I added an inbound HTTPS (443) rule to the security group. Port 80 stays open because the certificate authority checks the domain through it.

- **3. nginx:** I set `server_name askforhelp.duckdns.org;` in the nginx configuration and reloaded nginx.
- **4. Certificate:** I installed certbot and requested a free Let's Encrypt certificate. Certbot edits the nginx configuration to use it and sets up automatic renewal.

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d askforhelp.duckdns.org
```

Result: `https://askforhelp.duckdns.org` opens without problems and the chat works over a secure connection. The server is reachable through the domain name and through its public IP address (16.170.76.172 at the time of writing). After the assignment is graded, the instance and the Elastic IP should be deleted to avoid costs.

4.5 Problems and solutions in the finishing phase

6. The page was shown twice with junk text in the middle	
Problem	After updating the design, the browser showed two chat windows next to each other with the text `EOFcat > index.html << 'EOF'` between them.
Cause	The same paste problem as in problem 2: a second copy of the whole page was appended to index.html after the first closing </html> tag. The code was fine, only the file was wrong.
Solution	I deleted everything after the first `</html>` with one sed command, verified with `grep -c "<html" index.html` (must be 1) and reloaded the browser with Ctrl+F5.

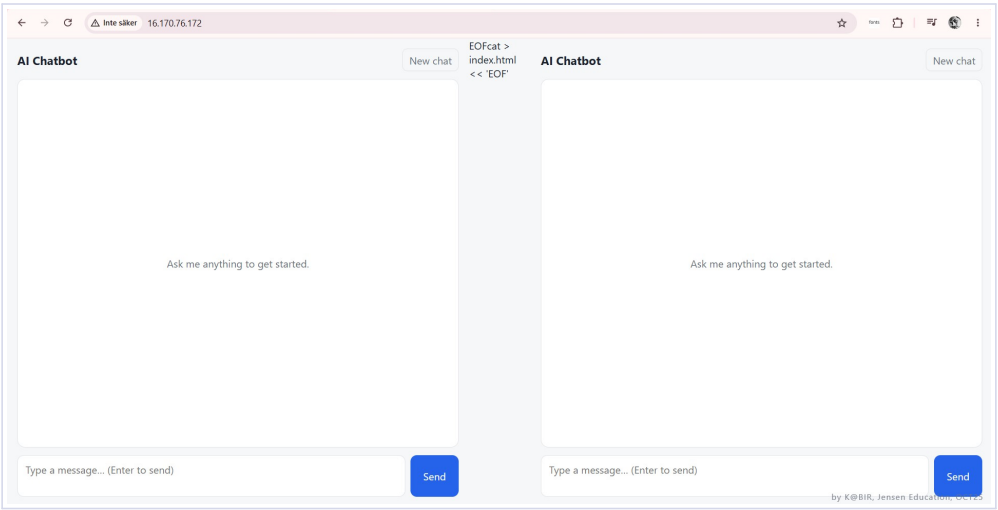


Figure 4. The corrupted index.html: two copies of the page and the leftover text in the middle.

7. The chatbot could not answer real-time questions	
Problem	Asked "how is the weather today", the bot answered that it has no access to real-time data.
Cause	A language model has no internet access, clock or location. My first version only forwarded messages to the model.
Solution	I implemented tool use (section 4.3): the model asks for a tool, the server runs it and returns the result, and the model writes the answer from fresh data. I tested the logic with a simulated model before deploying and then tested real questions on the server.

8. The bot kept talking about a "student project"	
Problem	Answers contained sentences such as "for your student project".
Cause	My default system prompt said "an assistant on a student project website", and the model followed that instruction.
Solution	I rewrote the system prompt (general helper, same language as the user, source names for web answers) and restarted the service.

9. HTTPS did not work, only HTTP	
Problem	Opening the site with https:// failed; only http:// worked, and the browser showed "Not secure".
Cause	The server only listened on port 80. HTTPS needs port 443 and a certificate, and a certificate cannot be issued for a bare IP address, only for a domain name.
Solution	I created the free domain `askforhelp.duckdns.org` at DuckDNS and pointed it to the Elastic IP, opened port 443 in the security group, set the server_name in nginx and requested a free Let's Encrypt certificate with certbot (section 4.4). The site now opens at https://askforhelp.duckdns.org without problems.

5. Phase 4: Summary of problems and solutions

#	Problem	Cause	Solution
1	Too many options at the start	Open assignment, no criteria	Compared options on 3 criteria; chose a chatbot on Bedrock + EC2
2	Unclear "Level 3" requirement	Short task text	Broke it into server, web server software and public IP
3	Model availability unclear	IDs depend on region and account	Checked console, tested a real call early
4	No connection to the instance	No public IP assigned	Allocated and attached an Elastic IP
5	Corrupted files after pasting	Several heredoc blocks pasted without waiting	Repaired with printf/sed; paste one block at a time and verify with tail/py_compile
6	boto3 not found	pip failed on the corrupted requirements.txt	Fixed the file and reinstalled
7	Variable not set	Typed < > placeholders literally	Removed brackets and checked with echo
8	Invalid model ID / access denied	Wrong ID; model not enabled for the account	Used an inference-profile ID from the console and tested with curl; kept the ID in an environment variable
9	Page shown twice with junk text	Duplicate paste after </html>	Deleted the extra part with sed; verified with grep
10	No real-time information	Model has no internet access	Added tool use: time, weather, web search
11	Unwanted "student project" wording	System prompt text	Rewrote the system prompt
12	HTTPS not working	Only port 80; no certificate for an IP	Free DuckDNS domain, port 443 opened, Let's Encrypt certificate with certbot

Main lesson from the problems: most delays came from small mistakes while moving code to the server, not from the AI itself. Checking every file after creating it, testing one layer at a time (model with curl, then server, then browser) and keeping settings in environment variables made the problems quick to find and fix.

6. Publishing the project on GitHub

The project is saved with Git and shared on GitHub, so it works as a backup and as a portfolio. The repository contains the code, deployment files and this documentation. Secrets are never committed: there are no API keys in the project, and the `.gitignore` file excludes `.pem` key files, `.env` files and the virtual environment.

Steps

1. Install Git and create a GitHub account. **2.** On GitHub choose "New repository", give it a name and click "Create repository". **3.** In the project folder run:

```
git init
git add .
git commit -m "First commit: AI chatbot on AWS Bedrock"
git remote add origin https://github.com/<username>/<repository>.git
git branch -M main
git push -u origin main
```

For later changes:

```
git add .
git commit -m "Describe what you changed"
git push
```

Common GitHub problems

Problem	Solution
"Permission denied" when pushing	Check that you are logged in with the right account; use a personal access token instead of a password when Git asks for one.
Wrong branch name (main / master)	Run `git branch -M main`.
Changes do not appear on GitHub	Remember `git push` after the commit.

7. Conclusion

What I achieved

- A working generative-AI chatbot where every answer comes from a Claude model on AWS Bedrock.
- A production-style deployment on AWS EC2: nginx, gunicorn and systemd, reachable through a public IP address.
- A free domain name with a trusted HTTPS certificate: https://askforhelp.duckdns.org.
- Secure access to the AI through an IAM role, with no keys stored on the server.
- Live information (date and time, weather, web search) through tool use, so the chatbot can answer current questions.
- A polished, responsive interface with light and dark themes and my own branding.
- Documentation of the whole process, including twelve problems and how they were solved.

What I learned

- How to plan a cloud project in small steps and verify each step before continuing.
- How AWS pieces fit together: regions, IAM roles, security groups, Elastic IPs, EC2 and Bedrock inference profiles.
- How a web application is served in production (reverse proxy, application server, service manager).
- How language models work in an application: system prompts, conversation history, and tool use for current data.
- Troubleshooting on a Linux server: reading error messages, logs (`journalctl`), and checking files before running them.
- How a domain name (DNS) and an HTTPS certificate work together with a web server.
- Using Git and GitHub to save and share a project.

What could be improved

- **Protection against misuse:** the page is public and every message costs money on Bedrock, so login, rate limiting or a usage budget alarm would be needed in real use.
- **Tighter permissions:** limit the IAM policy to the specific model instead of all Bedrock resources.
- **Streaming answers** (`converse_stream`) so text appears while it is generated, and Markdown formatting in answers.
- **Saved conversations** (for example in DynamoDB) and more tools.
- **Automation:** deploy with a script or infrastructure as code instead of typing commands, which would have avoided the paste errors.

The work resulted in a working, deployed solution and gave me practical experience in planning, cloud development, problem solving and deployment.